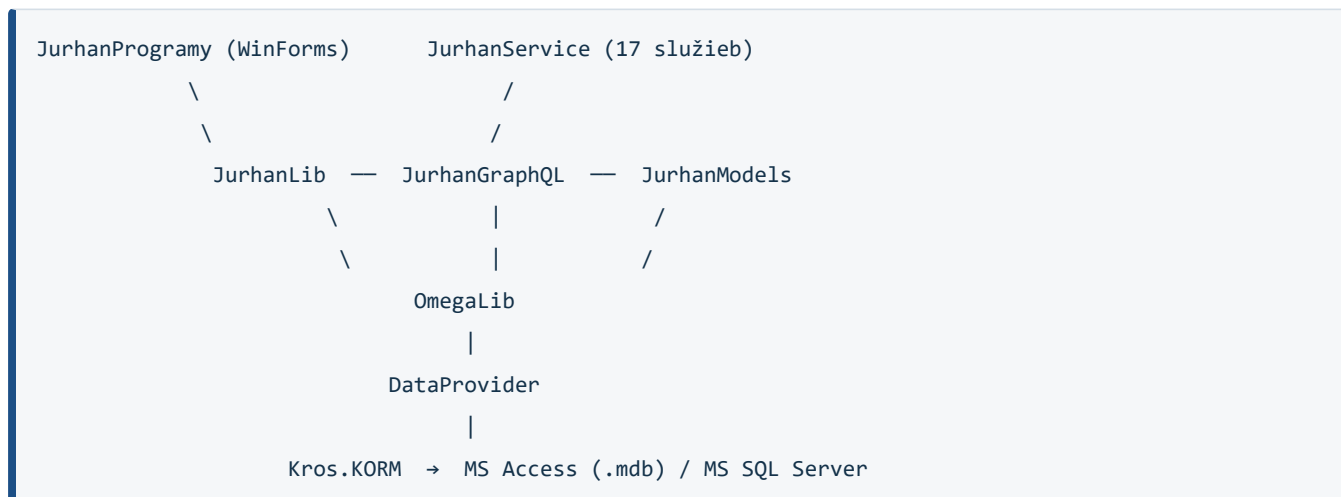


DataProvider — programátorská príručka

1. Zaradenie v ekosystéme

`DataProvider` je najnižšia vrstva celého riešenia Jurhan. Nad ňou stojí `OmegaLib` (modely + repozitáre nad tabuľkami Omegy), nad tou `JurhanModels` / `JurhanGraphQL` / `JurhanLib` a úplne hore desktopové programy (`JurhanProgramy`) a Windows služby (`JurhanService`).



- **Solution:** `DataProvider\DataProvider.sln`
- **Projekt:** `DataProvider\DataProvider\DataProvider.csproj`
- **Target:** `net10.0` (jediný projekt v riešení, ktorý *nie je* viazaný na Windows Forms)
- **Výstup:** `DataProvider.dll`

2. Verejné API

`IDataProvider`

Rozširuje `Kros.KORM.IDatabase`. Okrem KORM-ovského `Query<T>()` pridáva „surové“ ADO.NET operácie, ktoré Omega potrebuje pre SQL písané ručne:

Člen	Význam
<code>ExecuteCommand(query[, params])</code>	DML príkaz, vráti počet zmenených riadkov
<code>ExecuteReader(query[, params])</code>	<code>IDataReader</code> nad výsledkom
<code>ExecuteScalar(query)</code> / <code>ExecuteScalar<T>(query)</code>	prvá bunka prvého riadku
<code>GetSharedFolder()</code>	zdieľaný adresár Omegy (zámky, autocounter, prihlásení užívateľa)
<code>IsMsSql()</code>	rozlíšenie backendu — používa sa všade, kde sa SQL líši

Implementácie

- `DataProviderBase` — spoločná logika, cache otvorených providerov, mapovanie KORM.

- `AccessDataProvider` — `Microsoft.ACE.OLEDB.12.0` nad `.mdb`.
- `SQLDataProvider` — `System.Data.SqlClient` nad MS SQL.
- `oMsAccessQueryProvider` / `oSqlServerQueryProvider` — KORM query providery; sem patria odlišnosti v generovaní SQL (TOP/LIMIT, funkcie nad dátumami, spájanie reťazcov).
- `QueryProviderFactory` — výber providera podľa typu spojenia.
- `DefaultDirectoryInitializer` / `IDirectoryInitializer` — príprava pracovných adresárov.

3. Ako sa provider získava

Priamo sa `DataProvider` takmer nikdy neinštaluje. Volajúci vždy ide cez `OmegaLib.Repository.PripojeneFirmy`:

```
var pripojeneFirmy = new PripojeneFirmy(Constants.PathOmega);
pripojeneFirmy.Nacitaj(); // prečíta zoznam firiem z omega.mdb
pripojeneFirmy.OtvorPripojenuFirmu(idFirmy); // otvorí konkrétnu firmu/rok
IDataProvider dp = pripojeneFirmy.dataProvider; // až toto je DataProvider
```

`PripojeneFirmy` podľa príznaku `firma.SqlServer` zavolá `OpenSQLDb(...)` alebo `OpenAccessDb(...)` a výsledný provider uloží do `dataProvider`.

4. Cache spojení a súbežnosť

`DataProviderBase` drží **statickú** cache (`ConnectionList`) párov `connectionString + IDbConnection` → `IDataProvider`, aby sa pre to isté spojenie nevytváral nový provider. Cache je chránená zámkom (`lock`), pretože od prechodu na .NET 10 beží `JurhanGraphQL` ako in-process DLL a providery sa môžu vytvárať súbežne z viacerých vlákien (služba + GraphQL volania v jednom procese).

Pozor pri úpravách: cache je statická a proces služby beží dni. Akýkoľvek stav, ktorý sem pridáte, prežije všetky behy — nesmie závisieť od „aktuálneho behu“.

5. Bitness a prerekvizity

- Procesy sú **x64** (na .NET 10 už neexistuje x86 Jet 4.0, ktorý bol súčasťou Windows).
- Pre `.mdb` musí byť nainštalovaný **Microsoft Access Database Engine 2016 Redistributable (x64)**. Bez neho padne až za behu na `provider is not registered on the local machine`.
- Pre SQL Server nie je potrebná žiadna doinštalácia.

6. Build a nasadenie

```
dotnet build C:\Projekty\Private\DataProvider\DataProvider.sln -c Release
```

Knižnica sa nenasadzuje samostatne — kopíruje sa ako `DataProvider.dll` do výstupu každého programu/služby cez publish skripty `JurhanService\Deploy\Publish-AllJurhanServices.ps1` a `Publish-AllJurhanProgramy.ps1`.

7. Na čo si dať pozor pri ďalšom vývoji

1. **Nezavádzať rozdiely medzi backendmi mimo query providerov.** Ak niekde potrebujete `IsMssql()` vo vyššej vrstve, je to signál, že rozdiel patrí sem dole.
2. **Kros.KORM verzia** — celý ekosystém je viazaný na verziu KORM dodávanú s Omegou (`C:\Omega`). Pridanie `PackageReference` na inú verziu spôsobuje `MissingMethodException` za behu (historicky sa to stalo v `ParovanieKariet`).
3. `ExecuteScalar` **vracia** `Nullable<T>` — pri prázdnej tabuľke je to `null` , nie `0` . Nezakrývajte to `catch { return 0; }` , ako sa to kedysi robilo v `Repository.DajMaxId` — viedlo to ku kolíziám ID.
4. Nikdy neskrývajte chyby DB. Výnimka musí prebublať až do `ServiceJurhan.OnElapsedTime` , ktorý ju zalogue a pošle e-mail.